

Dator Lab - Fysikens matematiska metoder

Lukas Ejvinsson (luej0002@student.umu.se)

Tim Tiensuu (titi0440@student.umu.se)

Elliot Åkesson (elak0044@student.umu.se)

1 Uppgift 1

I uppgift 1 skulle vi skapa två matlab funktioner, **mydft.m** och **myidft.m**. Dessa funktioner skulle beräkna den diskreta fouriertransformen och den inversa diskreta fouriertransformen. Sedan skulle vi beräkna fouriertransformen och den inversa fouriertransformen för vektorn $y = (0, 3, -2\sqrt{3}, 6, 2\sqrt{3}, 3)$.

För vektorn $y = (0, 3, -2\sqrt{3}, 6, 2\sqrt{3}, 3)$ ger funktionen $z = \text{mydft}(y)$ att $z = (2, -\frac{1}{2} + i, \frac{1}{2} - i, -2, \frac{1}{2} + i, -\frac{1}{2} - i)$ och funktionen $w = \text{myidft}(y)$ ger $w = (12, -3 - 6i, 3 + 6i, -12, 3 - 6i, -3 + 6i)$.

2 Uppgift 2

I uppgift 2 skulle vi först skapa funktionen **myfouriercoeff.m**. Och med denna funktion skulle vi beräkna fourierkoefficienterna till olika funktioner efter att vi hade satt in dessa funktioner i **mydft.m** i uppgift a) så skulle vi beräkna fourierkoefficienterna till en funktion. och i uppgift b) skulle vi plotta fourierkoefficienterna vi fick från vår funktion **myfouriercoeff.m** och plotta dem med dem exakta fourierkoefficienterna för en funktion $g(x)$.

2.1 a)

Beräkna de exakta fourierkoefficienterna för den 2π - periodiska funktionen $f(x) = 3 - 2\cos(15x) + 4\sin(20x)$.

Vi kan direkt utläsa att $a_0 = 3, a_{15} = -2, b_{20} = 4$, utifrån den generella formen för fourierserie.

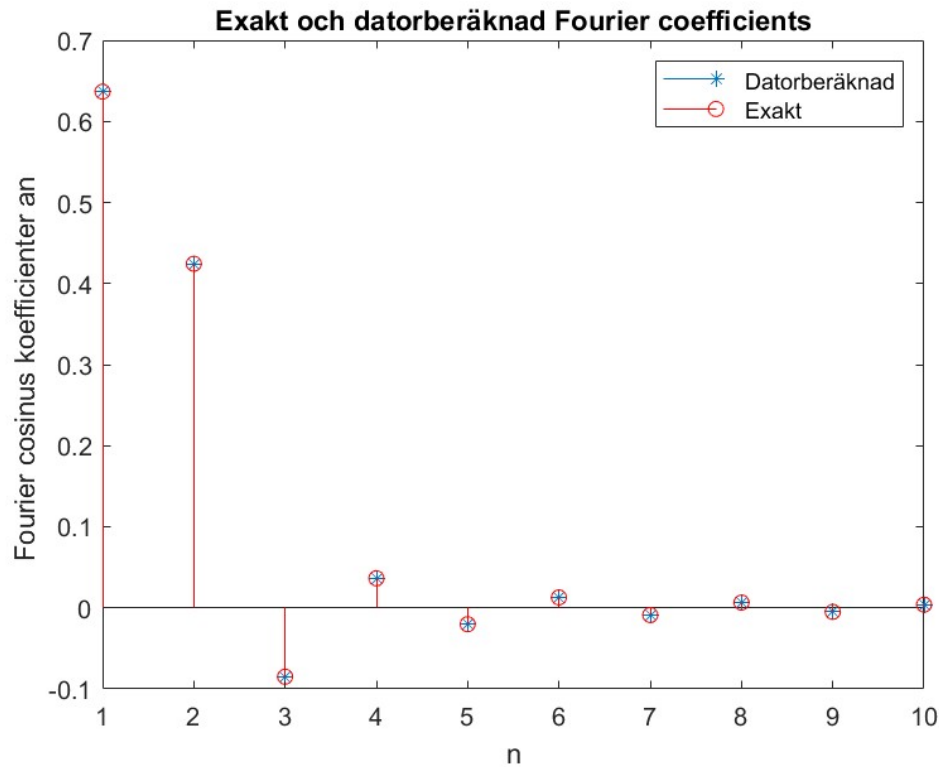
Fourierkoefficienterna för $N = 2^5$ respektive $N = 2^8$ beräknat med dator

För $N = 2^5$ får vi att $a_0 = 3, a_{15} = -2, b_{12} = -4$ och övriga värden är approximativt 0 ($< e - 13$). För $N = 2^8$ får vi att $a_0 = 3, a_{15} = -2, b_{20} = 4$ och övriga värden är approximativt 0 ($< e - 13$).

För $N = 2^8$ ser vi att Fourierkoefficienterna har samma värden som de exakt beräknade. För $N = 2^5$ ser vi att vi bara får 16 koefficienter vilket betyder att vi aldrig kommer till b_{20} . Vi får då att $b_{16-4} = -4$ istället för $b_{20} = 4$. Anledningen till detta är den komplexa symmetrin i vår diskreta fouriertransform.

2.2 b)

Beräkna med funktionen **myfouriercoeff.m** fourierkoefficienterna för den jämna funktionen $g(x) = |\cos(x)|$ definierad på intervallet $[0, 2\pi]$. Jämför genom att plotta i samma graf, med exakta fourierserien $g(x) \sim \frac{2}{\pi} - \frac{4}{\pi} \sum_{n=1}^{\infty} \frac{(-1)^n \cos(2nx)}{4n^2 - 1}$. Vi valde $N = 2^8$ punkter på intervallet genom att testa oss fram. Vid ett lägre värde på N blev skillnaden mellan den datorberäknade fourierserien och den exakta fourierserien stor. Om vi istället tar ett större värde på N påverkas serien knappt, medan tiden som beräkningen tar ökar avsevärt.



Figur 1: Antal datapunkter: $N = 2^8$

Vi ser i figuren att värdena för de exakta och datorberäknade fourierkoefficienterna är approximativt identiska.

3 Uppgift 3

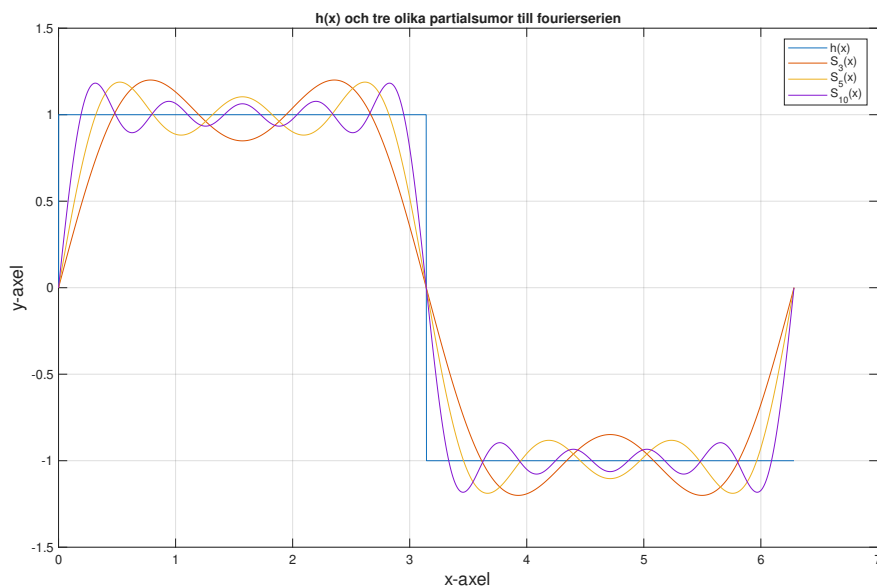
I uppgift 3 skulle vi undersöka ett fenomen som kallas Gibbs fenomenet. Vi skulle undersöka vad som händer med en fourierserie och vår datorberäknade fourierserie vid en diskontinuitet. Vi gjorde det till en funktion $h(x)$. Vi jämförde en partiella fourierserier med olika antal termer och undersökte det största felet mellan funktionen $h(x)$, de datorberäknade fourierserierna och den exakta fourierserien till $h(x)$.

3.1 a)

Vi har den 2π -periodiska funktionen

$$h(x) = \begin{cases} 1 & 0 < x < \pi \\ -1 & \pi < x < 2\pi \\ 0 & x = 0, x = \pi \end{cases}$$

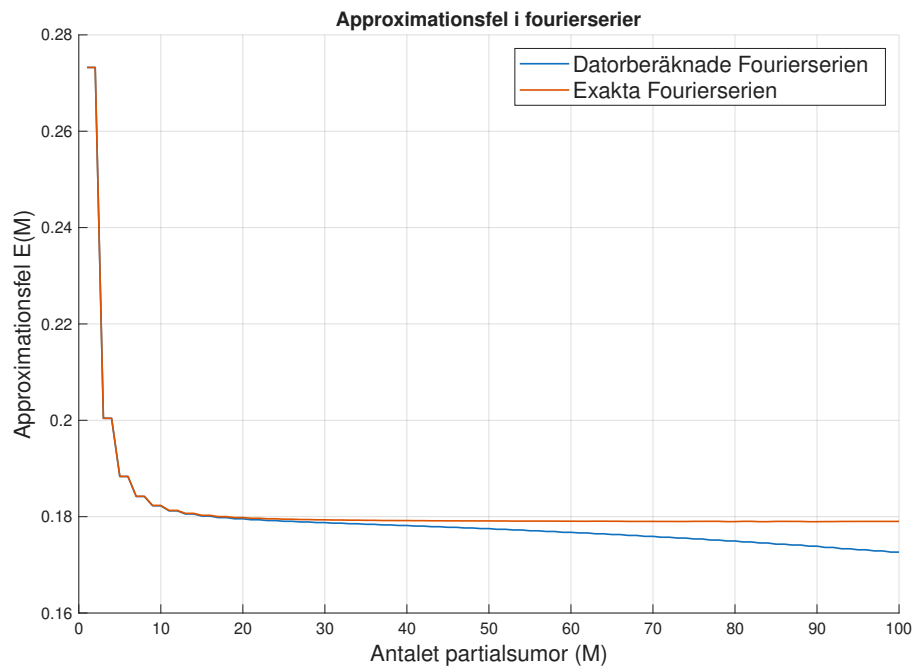
Vi beräknar med funktionen **myfouriercoeff.m** fourierkoefficienterna och plottar partialsummorna för fourierserien för $M = (3, 5, 10)$ på intervallet $[0, 2\pi]$, tillsammans med $h(x)$.



Figur 2: $h(x)$ med partialsummorna, $S_3(x)$, $S_5(x)$, $S_{10}(x)$

Vi kan i figur 2 se att alla tre partialsummorna till $h(x)$ överskjuter $h(x)$ med ca $0.2y$. Däremot så håller sig kurvan med mest partialsummorna närmast $h(x)$. Vilket stämmer bra överens med teorin om fourierserier.

3.2 b)



Figur 3: Approximationsfelet mellan $h(x)$, Datorberäknade Fourierserien och den exakta fourierserien kring $x = \pi$. Här är $N = 2^{10}$

Vi ser i figur 3 att approximationen av koefficienterna för fourierserien $h(x)$ överskjuter mindre och mindre med fler partialsummor. Med de exakta koefficienterna verkar istället överskjutningen plana ut och blir inte mindre. Detta är inget magiskt utan nämligen Gibbs fenomen[1]. Gibbs fenomen säger att när man närmar sig en diskontinuitet med en fourierserie, får man alltid en överslag, oavsett hur många partialsummor man adderar. Och att felet i approximationen till slut stabiliserar sig vid $\approx 9\%$ av storleken av ”hoppet”.

När den exakta fourierserien närmar sig diskontinuiteten vid $x = \pi$, så överskjuter fourierserien det faktiska värdet av funktionen. Desto mer partialsummor man adderar, ser man att felet i uppskattningen mellan fourierserien och funktionen, stabiliserar sig kring 9% av hoppets värde. Och eftersom hoppet i $h(x)$ kring $x = \pi$, är av storlek 2. Och felet stabiliserar sig kring 9% enligt Gibbs fenomen, så förklarar det bra varför felet i figur 3 stabiliserar sig kring 0.18 för den exakta fourierserien.

Kollar vi på den datorberäknade fourierserien ser det nästan ut som att den approximerar $h(x)$ bättre än den exakta fourierserien i figur 3. Men när man höjer N till N^{12} , överlappar fourierserierna varandra och felet stabiliserar sig kring 0.18. På grund av att vi hade för få datapunkter verkade det som att Gibbs fenomen inte stämde. Dock kan vi inte säga att den datorberäknade fourierserien är bättre, eftersom den istället approximerar funktionen sämre vid andra punkter.

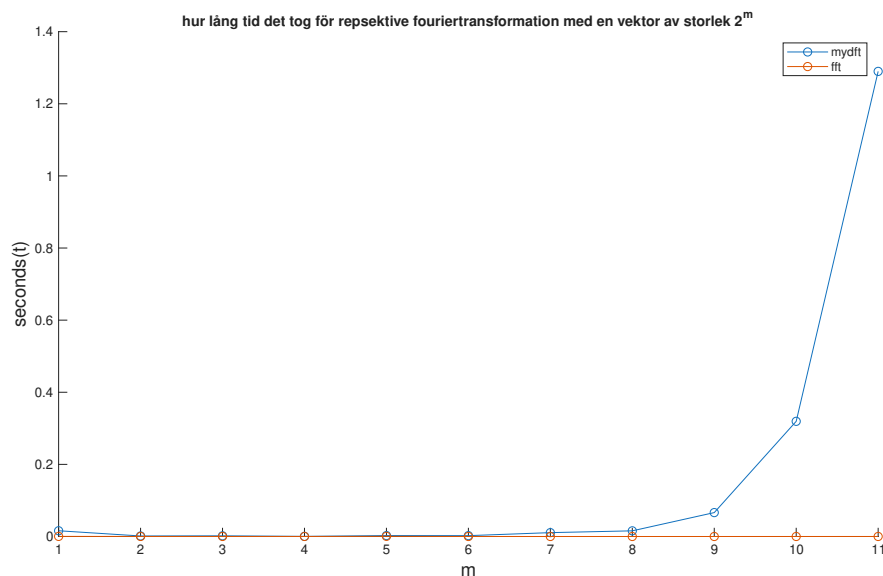
4 Uppgift 4

Uppgift 4 handlade om den snabba fouriertransformen, och den snabba inversa fouriertransformen. I uppgift 4a) skulle vi jämföra vår fouriertransformation mot matlabs inbyggda

fouriertransformation. Sedan i uppgift (b) - (c) skulle vi använda matlabs snabba fouriertransformation och snabba inversa fouriertransformation för att se användbara appliceringar av fouriertransformer i verkligheten.

4.1 a)

Vi skulle undersöka hur lång tid det tog för vår fouriertransform(**mydft**) och den snabba fouriertransformen(**fft**). Sedan skulle vi jämföra dem och se vilken som var snabbast med vektorer av storlek 2^m där $m = 1, 2, \dots, 11$. För att göra dessa undersökningar skapade vi slumpade vektorer av storlek $(1, 2^m)$. sedan testade plotta dem mot varandra och fick figur 4. I figur 4 kan vi tydligt se att den snabba fouriertransformen(**fft**) är betydligt snabbare då vektorn blir större. Men även om man kollar för mindre vektorer är den snabba fouriertransformationen bättre.

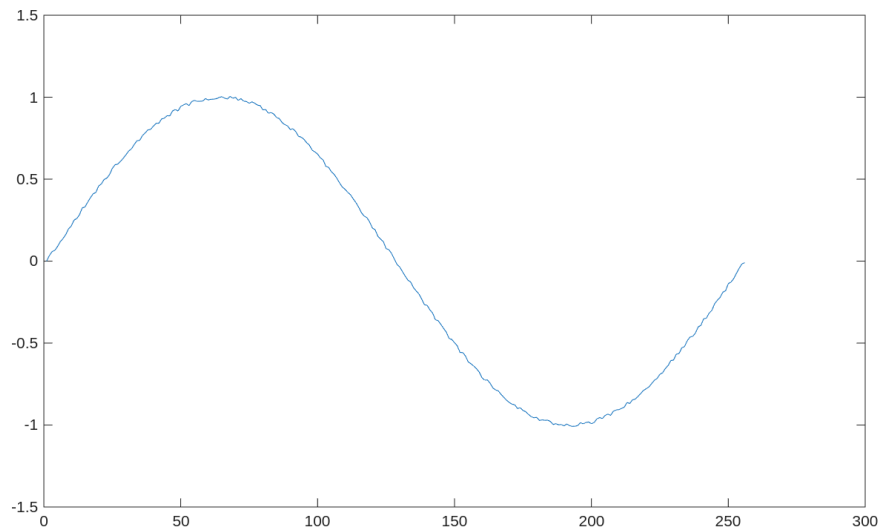


Figur 4: Hur snabb fouriertransformationerna **fft** och **mydft** var på att transformera en vektor av storlek 2^m

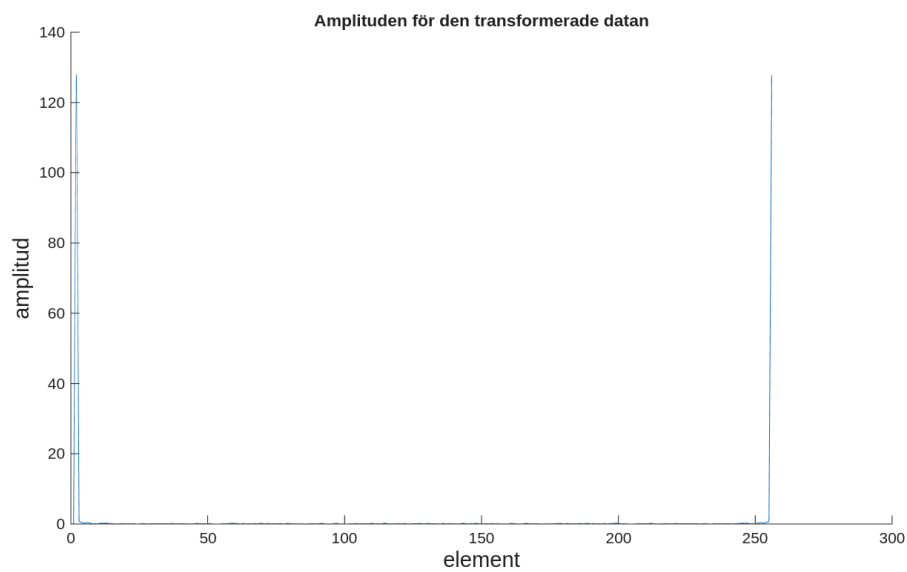
4.2 b)

Vi skulle ladda ner data från canvassidan till denna kurs. Sedan skulle vi applicera den snabba fouriertransformen(**fft**) och den snabba inversa fouriertransformen(**ifft**) för att kunna ta bort vis "brus" data från original datan. Original data plottades i figur 5.

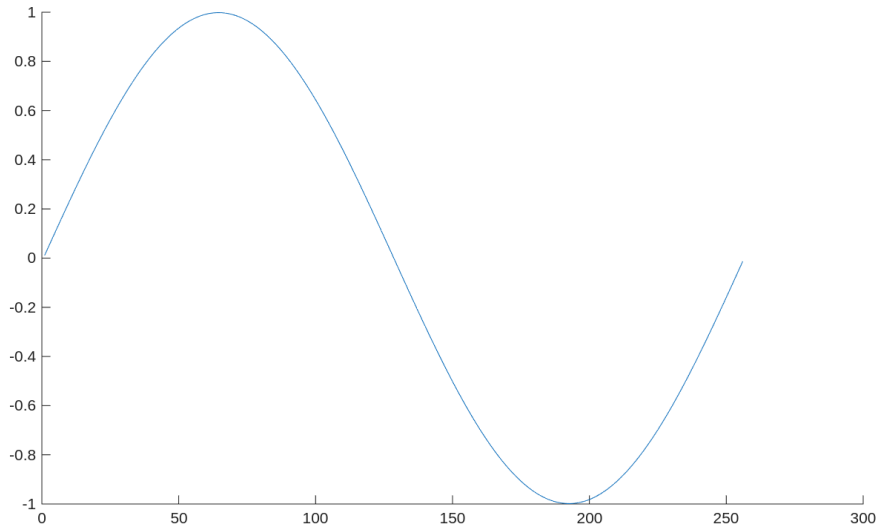
Vi började med att applicera den snabba fouriertransformen på datan. Sedan tog vi absolutbelopet på den transformerade datan för att se vilken amplitud datan ligger på. Vi plottade absolutbelopet av den transformerade datan i figur 6. Där kunde vi se att den mesta av "brus" datan låg väldigt nära noll medans de mesta av datan hade betydligt större amplitud. Detta ledde till att vi valde $wcut = 1$. Alltså vi tog bort all data som hade amplitud mindre än ett. Sedan applicerades den inversa snabba fouriertransformen på den filtrerade datan och då fick vi 7. Studerar man figur 7 kan man se att kurvan är betydligt mycket slätare än i figur 5.



Figur 5: Den ofiltrerade datan.



Figur 6: absolutbelopet av den transformerade datan.



Figur 7: datan efter att man har tagit bort "brus" datan, notera att kurvan är betydligt mer slätt än i den ofiltrerade datan.

4.3 c)

I Denna uppgift skulle vi jämföra ljudkvalitet och storlek på en fil beroende på hur mycket ljudet man filtrerar bort. För att göra det så måste man välja ett w_r , den procentuella delen av maxvärdet/max - amplituden. Sedan filtrerar man bort all ljud data som är mindre än w_r . Sedan sparar man ner all filtrerad data i en ny fyll. Efter detta kan man jämföra ljudkvalitén och hur mycket mer komprimerad datan är före och efter att man har applicera den snabba fouriertransformen på den och sorterat bort viss data. Och hur mycket sämre ljudkvalitén har blivit. Vi skapade en skala på ljudkvalitén där 1 är sämst och 10 är lika bra som original ljudet. För att få ett mått på hur mycket mindre filen har blivit tar man bara att delar original filens storlek med den nya filens storlek och sparar det i en variabel som heter CompRatio.

Vi valde ljudfilerna "Train" och "Gong". Vi undersökte först båda filerna när man sate $w_r = 0.5$ av den största amplituden. Vi märkte i båda fallen att data filen vart betydligt mycket mindre, men ljudkvalitén vart bedrövlig. Vi testade även $w_r = 0.1$ och $w_r = 0.03$. Resultaten går att utläsa ur tabell 1 och tabell 2. Det som vart tydligt var att om man valde ett litet w_r kunde man minska storleken på filen ganska avsevärt utan att man i princip förlorade någon ljudkvalitet. Den här metoden kan nog vara väldigt bra på längre ljudfiler då man kan kapa storleken på ljudfilen ganska drastiskt genom att bara ta bort lite bakgrundsljud.

Tabell 1: Train

	CompRatio	Kvalitet
$w_r = 0.5$	772.840	1
$w_r = 0.1$	72.9094	5
$w_r = 0.03$	29.0979	9

Tabell 2: Gong

	CompRatio	Kvalitet
$w_r = 0.5$	578.3761	1
$w_r = 0.1$	12.8896	6
$w_r = 0.03$	3.1131	9

Referenser

- [1] Wikipedia contributors. Gibbs phenomenon. https://en.wikipedia.org/wiki/Gibbs_phenomenon, 2025. Accessed: 2025-10-30.

Appendix

mydft.m

```
1 function z = mydft(y)
2 % Compute the DFT of a vector y of length N
3
4 N = length(y);
5 j = 0:(N-1); % row vector containing the indexes needed for the
    sum
6 n = j'; % column vector containing indexes, needed to use
    matrix multiplication instead of for loops
7 omega = exp (2*pi*1i/N); % 1i specifies the imaginary unit
8
9 % Uses element wise multiplication in combination with matrix
10 % multiplication to sum over j and output a row vector z_n
11 z = (1/N) .* y*omega.^(-n*j);
```

myidft.m

```
1 function y = myidft(z)
2 % Compute the Inverse DFT of a vector y of length N
3
4 N = length(z);
5 j = 0:(N-1); % row vector containing the indexes needed for the
    sum.
6 n = j'; % column vector containing indexes, needed to use
    matrix multiplication instead of for loops
7 omega = exp (2*pi*1i/N); % 1i specifies the imaginary unit.
8
9 % Reverses what mydft.m does to output the row vector y_n
10 y = z*omega.^(n*j);
```

myfouriercoeff.m

```
1 function [a0,a,b] = myfouriercoeff(z)
2 % -In data -
3 % z: vector containing complex Discrete Fourier transform
4 % - Out data -
```

```
5 % a0: the coefficient a_0
6 % a: vector with the Fourier coefficients a_n
7 % b: vector with the Fourier coefficients b_n
8
9 N = length(z); % Compute the length of vector z
10
11 % remove first element z(1) from z-vector and pick out the
    first half
12 % of the z-vector (excluding the first 0 frequency, so from 2
    to
13 % N/2+1) since it is complex symmetric. i.e. c_n = conj(c_(N-n)
    ). Also
14 % take the floor to make it work for odd lengths for the input
    vector.
15 c = z(2:floor(N/2)+1);
16
17 % The first value is a0, this is the amplitude of the 0
    frequency.
18 a0 = z(1);
19
20 % Because of the complex symmetry we can use this
    computationally
21 % cheaper method to calculate the fourier coefficients.
22 a = c + conj(c);
23 b = 1i * (c - conj(c));
```

task1.m

```
1 % Task 1
2 % Create mydft.m and myidft.m and use them.
3 clear
4 close all
5
6 %% Test of functions
7
8 y1 = [8 12 -4 4]
9 z1 = mydft(y1)
10
11 isequal(round(myidft(z1)),round(y1)) % Should always return
    true!
12
13
14 %% Results to include in report
15
16 y2 = [0, 3, -2*sqrt(3), 6, 2*sqrt(3), 3]
17 z2 = mydft(y2)
18 w2 = myidft(y2)
```

task2.m

```
1 clear
2 close all
3 % a)
4 % Find, by inspection, the exact Fourier series
```

```
5 % coefficients ai and bi of the 2*pi-periodic function
6
7 %f(x) = 3 - 2 cos(15x) + 4 sin(20x)
8
9 %a_0 = 3
10 %a_15 = -2
11 %_b20 = 4
12
13 % Compute the DFT with N = 32
14 N_1 = 2^5;
15 x1 = 2*pi*(0:N_1-1)/N_1; % No for loop needed :)
16 y1 = 3 - 2*cos(15*x1) + 4 * sin(20*x1);
17 z1 = mydft(y1);
18
19 % Use myfouriercoeff to get calculated coefficients
20 [a0_1, a_1, b_1] = myfouriercoeff(z1);
21
22
23 % Compute the same DFT but with N = 256
24 N_2 = 2^8;
25 x2 = 2*pi*(0:N_2-1)/N_2;
26 y2 = 3 - 2*cos(15*x2) + 4 * sin(20*x2);
27 z2 = mydft(y2);
28 [a0_2, a_2, b_2] = myfouriercoeff(z2);
29
30 % Values to include in report
31 fprintf('Exact coefficients: a_0: %d, a_15 %d, b_20 = %d \n',
32         ,3,-2,4)
33 fprintf('Calculated with N=32, a_0: %d, a_15 %d, b_12 = %d \n',
34         ,round(a0_1) ...
35         ,round(a_1(15)),round(b_1(12)))
36 fprintf('Calculated with N=256, a_0: %d, a_15 %d, b_20 = %d \n',
37         ,round(a0_2) ...
38         ,round(a_2(15)),round(b_2(20)))
39
40 %%
41
42 % b)
43 % The function g(x)
44 g = @(x) abs(cos(x));
45
46 % Use appropriate N
47 N_g = 2^8; % include in report!
48
49 % define x for the interval [0,2pi]
50 x_g = 2*pi*(0:N_g-1)/N_g;
51 z_g = mydft(g(x_g));
52 [a0_g, a_g, b_g] = myfouriercoeff(z_g);
53 % in order to compare with exact coefficients we only want the
54 even
55 % coefficients.
56 a_g = a_g(2:2:end);
```

```
54
55
56 % Plot against exact coefficients
57 a0exact = 2/pi;
58 aexact = (-4*(-1).^(1:N_g/4))./(pi*(4*(1:N_g/4).^2 -1 ));
59 figure()
60 stem([a0_g a_g], '*')% plot computed Fourier cosine coefficients
61 hold on
62 stem([a0exact aexact], 'or')% plot exact Fourier cosine
    coefficients
63 xlim([1 10])
64 hold on
65 xlabel('n'); ylabel('Fourier cosine coefficients a_n ');
66 title('Exact and computed Fourier coefficients')
67 legend('Computed', 'Exact');
68
69 % Include in report
70 % N = 2^8
71 % Figure!
```

task3.m

```
1 clear
2 close all
3 % a)
4
5 N=2^10;
6 x = 2*pi*(0:N-1)/N;
7 h = hfun(x); % Defined at the bottom of this file
8
9
10
11 z = mydft(h);
12 [a0,a,b] = myfouriercoeff(z);
13
14 % Above you have computed the Fourier coefficients a0 , a_n ,
    and b_n of h(x)
15 t = linspace(0,2*pi,5000); %
16 y3=a0;
17 y5=a0;
18 y10=a0;
19
20 % I could not figure out a better way to do this then with 3
    for loops
21 for j=1:3 % The partial sum of the Fourier series with M+1
    terms
22     y3=y3+a(j)*cos(j*t)+b(j)*sin(j*t); % add a term of the
        Fourier series
23 end
24 for j=1:5 % The partial sum of the Fourier series with M+1
    terms
25     y5=y5+a(j)*cos(j*t)+b(j)*sin(j*t); % add a term of the
        Fourier series
26 end
```

```
27 for j=1:10 % The partial sum of the Fourier series with M+1
    terms
28     y10=y10+a(j)*cos(j*t)+b(j)*sin(j*t); % add a term of the
        Fourier series
29 end
30
31 %plot h together with several of the partial sums of the
    Fourier series
32 figure()
33 plot(t,hfun(t),t,y3,t,y5,t,y10)% (t,hfun(t)) is the exact
    function.
34 xlim([0 2*pi]) % plot from 0 to 2*pi
35 xlabel('x')
36 ylabel('h(x)')
37 legend('h(x)', 'S_3(x)', 'S_5(x)', 'S_{10}(x)')
38 title('Comparsion of the function h(x) and three partial
    fourier sums')
39
40
41 %%
42 % b)
43
44 t = linspace(0,pi,5002);
45 t = t(2:end-1); % exclude the points x=0 and x=pi
46
47 M = 1:100;
48
49 Yh = 0;
50
51 for n=M
52     Y = partialfourier(n,a0,a,b,t);
53     E(n) = max(Y-1);
54     Yh = Yh + (2/(pi*n))*(1-(-1)^n)*sin(n*t);
55     Eh(n) = max(Yh-1);
56 end
57
58 figure()
59 plot(M,E,M,Eh)
60 xlabel('M')
61 ylabel('E(M)')
62 legend('Error for computed coefficients', 'Error for exact
    coefficients')
63 title('Comparsion: Error of mydft and error of exact partial
    fourier sum')
64
65
66
67 % Function for h(x)!
68 function h = hfun(x)
69 %HFUN the 2-pi periodic function required for task 3
70
71 % Uses the modulus to make the function periodic.
72 rest = mod(x,2*pi);
```

```
73
74 %Preallocate
75 h = zeros(1,length(x));
76
77 h(rest > 0 & rest < pi) = 1;
78 h(rest > pi & rest < 2*pi) = -1;
79
80 end
```

task4.m

```
1 % a)
2 clear
3 close all
4
5 N = 2.^(0:11);
6 t_f = zeros(1,length(N));
7 t_matlab_f = zeros(1,length(N));
8
9 for i = 1:length(N)
10     y = rand(1,N(i));
11     f = @() mydft(y);
12     matlab_f = @() fft(y);
13
14     % Measure execution time for both DFT implementations
15     t_f(i) = timeit(f);
16     t_matlab_f(i) = timeit(matlab_f);
17
18 end
19 figure()
20 plot(t_f)
21 hold on
22 plot(t_matlab_f)
23
24 %%
25 clear
26 close all
27
28 % b)
29
30 fid = fopen('filtre.data','r');
31 Y = fscanf(fid,'%f',[1 inf]);
32 fclose(fid);
33 figure(), plot(Y)
34 Z = fft(Y);
35
36 plot(Y)
37
38 wcut = 1;
39 N = length(Z);
40 for j = 1:N
41     if (abs(Z(j)) < wcut)
42         Z(j) = 0;
43     end
```

```
44 end
45
46 newY = ifft(Z);
47
48 figure(), plot(newY)
49
50 %%
51 close
52 clear all
53 % c)
54
55 load('train')
56 sound(y,Fs)
57 Y = fft(y);
58 W = Y;
59 M = max(abs(Y));
60 omega_r = 0.8;
61 N = length(y);
62 for j = 1:N
63     if (abs(Y(j)) < omega_r * M)
64         W(j) = 0;
65     end
66 end
67
68 WS = sparse(W);
69 YS = sparse(Y);
70
71 before = whos('YS');
72 after = whos('WS');
73 comprRatio = before.bytes/after.bytes;
74 pause(4);
75 disp(comprRatio);
76 disp('Play compressed signal');
77 w = real(ifft(full(WS)));
78 sound(w,Fs)
```

partialfourier.m

```
1 function Y = partialfourier(M,a0,a,b,t)
2 %PARTIARFOURIER Compute the partial fourier for task 3
3
4 Y=a0;
5 if mod(M,1)==0 && M>0
6     for j=1:M% The partial sum of the Fourier series with M+1
7         terms
8             Y=Y+a(j)*cos(j*t)+b(j)*sin(j*t);% add a term of the
9             Fourier series
10    end
11 else
12    error('M is not an integer')
13 end
```